

Cost-of-Living Data Platform

Cloud Pipeline, Search Taxonomy, and Analytics API

Project Contributors

2026

Abstract

This report describes a cloud data platform for monitoring Australian cost-of-living pressure through public social discussion, media coverage and official CPI indicators. The system collects public records from Bluesky, Mastodon and GDELT GKG archive files, stores source-specific raw streams in Elasticsearch, normalises them into a unified raw index, processes text with topic classification and VADER sentiment, and serves analytics through a FastAPI REST API. ABS Monthly CPI observations are used as an official comparison signal rather than a causal label. The public version presents a single API serving path and uses Fission only for scheduled pipeline jobs, avoiding duplicate API deployments.

Contents

1	Scenario and Motivation	2
2	Data Sources and Search Terms	2
3	Architecture	3
3.1	Elasticsearch Data Model	4
4	API Design	4
5	Implementation Quality	4
6	Validation and Results	5
6.1	Topic and Sentiment Patterns	6
6.2	Official CPI Comparison	7
7	Limitations	7
8	Conclusion	8

1 Scenario and Motivation

Cost-of-living pressure affects household decisions around rent, groceries, fuel, energy, healthcare, transport, education and debt. Official indicators such as CPI provide a stable statistical view, but they are released on a schedule and do not directly show what people and media outlets are discussing day by day. Public social posts and media records are timely but noisy, inconsistent and hard to inspect manually at scale.

The platform therefore focuses on a repeatable monitoring workflow:

- collect public source records on a schedule;
- filter for Australian context and cost-of-living relevance;
- preserve raw records and processing state;
- process text into topics, sentiment labels and quality flags;
- expose chart-ready analytics through a REST API;
- compare selected discussion signals with official ABS CPI observations.

The aim is not to infer a representative measure of hardship. The aim is to build an inspectable data system that separates social discussion, media coverage and official indicators.

2 Data Sources and Search Terms

Table 1: Data sources

Source	Role	Interpretation
Bluesky	Public social discussion	Timely public posts, filtered for Australian context. Not population-representative.
Mastodon	Public social discussion	Public posts from multiple instances, useful as a second social signal.
GDELT GKG	Media coverage	Public 15-minute archive files used as media attention. It is not treated as public sentiment.
ABS Monthly CPI	Official indicator	Monthly official price data for comparison, not causal modelling.

The collection taxonomy is explicit and versioned in `data/cost_of_living_keywords.csv`. It includes housing, groceries, energy, fuel, transport, eating out, healthcare, home goods, education, inflation, wages and debt. Terms were selected to cover common Australian cost categories, official CPI-aligned categories and locally meaningful signals such as major supermarket names, public transport terms and healthcare references. Source-specific collectors follow the public Bluesky, Mastodon and GDELT interfaces [?, ?, ?], while official monthly indicators are obtained from ABS CPI documentation [?].

For social sources, a record must have a cost-of-living signal and Australian context. For GDELT, filtering is stricter because GKG records can contain metadata-heavy snippets. Incremental harvesting and historical backfill both use the same GKG archive processor: it reads the public master file list, downloads `.gkg.csv.zip` archives, verifies md5 checksums, extracts CSV rows, filters for relevant Australian cost-of-living records and bulk indexes matching rows. GDELT is analysed as media coverage, while Bluesky and Mastodon are grouped as social discussion.

3 Architecture

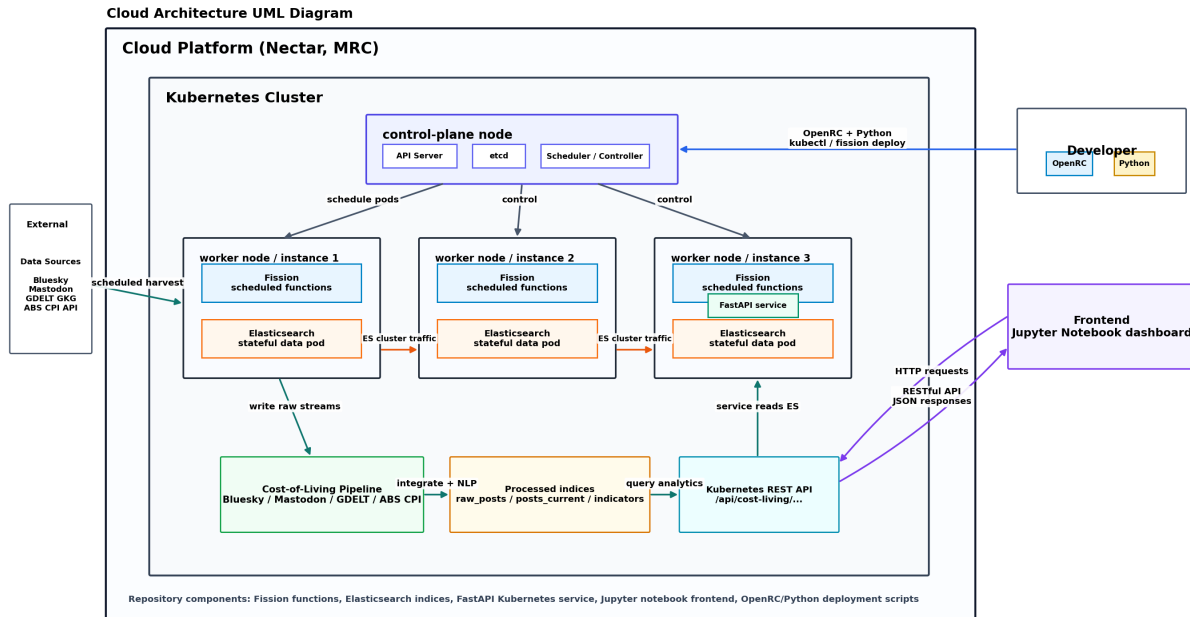


Figure 1: System architecture

The public architecture has one API layer. FastAPI serves all analytics endpoints. Fission is used for scheduled ingestion and processing jobs only. This avoids the operational ambiguity of deploying the same analytics API as both a monolithic service and separate HTTP functions. Kubernetes, Fission and Elasticsearch provide orchestration, scheduled function execution and search storage respectively [?, ?, ?].

Table 2: Runtime responsibilities

Component	Responsibility
Fission harvesters	Collect Bluesky, Mastodon, GDELT and ABS CPI data on schedules.
Raw integrator	Normalise source-specific records into the unified raw schema.
NLP processor	Claim pending records, clean text, classify topics, run sentiment and update processing state.
Elasticsearch	Store raw streams, raw processing state, processed documents, aliases and official indicators.
FastAPI	Serve REST endpoints for notebooks and dashboard clients.
Jupyter notebooks	Consume the API and render exploratory charts.

3.1 Elasticsearch Data Model

Table 3: Core indices and aliases

Index or alias	Purpose
<code>cost_living_bluesky_raw_stream</code>	Bluesky source-specific raw records.
<code>cost_living_mastodon_raw_stream</code>	Mastodon source-specific raw records.
<code>cost_living_gdelt_raw_stream</code>	GDELT source-specific raw records.
<code>cost_living_raw_posts</code>	Unified raw records and NLP processing state.
<code>cost_living_processed_posts</code>	Processed topic and sentiment documents.
<code>cost_living_posts_current</code>	Stable API read alias.
<code>cost_living_indicators</code>	ABS CPI observations.
<code>cost_living_monthly_topic_metrics</code>	Optional monthly rollup.

The NLP worker uses Elasticsearch as an inspectable work queue. Raw records start as pending. A worker atomically claims a record by moving it to processing, then writes a processed record and updates the raw status to processed, discarded or error. Stale processing records can be retried. This is simpler than operating a separate message broker and is appropriate for the batch-style workload, although it is not a general-purpose queue. Sentiment scoring uses VADER because it is lightweight and interpretable for short social text [?].

4 API Design

The API is read-only and exposes analytics resources under:

`/api/cost-living`

The application also accepts internal `/api/...` paths for local tests. Common filters include `source_group`, `platform`, `topic`, `start`, `end`, `period`, `quality` and `exclude_quality_flags`.

Table 4: Endpoint groups

Group	Examples
Health and pipeline	<code>/health</code> , <code>/pipeline/status</code>
Overview and trends	<code>/stats/overview</code> , <code>/trends/documents</code> , <code>/trends/categories</code>
Topic analytics	<code>/categories/counts</code> , <code>/categories/sentiment</code> , <code>/categories/share</code>
Data quality	<code>/data-quality/summary</code> , <code>/data-quality/comparison</code>
Media and platform views	<code>/media/coverage</code> , <code>/platforms/categories</code>
Official comparison	<code>/official/comparison</code>
Diagnostics	<code>/logs/errors</code>

This style is analytics-resource oriented rather than RPC-oriented command naming. The API uses GET because clients read aggregates and do not mutate platform state.

5 Implementation Quality

The codebase uses Pydantic settings, typed Python, explicit Elasticsearch mappings, a shared source registry and focused pytest coverage. The public version also removes duplicate Fission HTTP API functions so the

deployment story is consistent.

Important reliability choices include:

- stable ids for idempotent writes;
- separate raw and processed indices;
- explicit processing state fields;
- a stable processed-data alias for API reads;
- short API response caching to reduce repeated aggregation load;
- request logging in the FastAPI layer;
- Redis-backed job locks, lifecycle events and run identifiers for scheduled jobs;
- Kubernetes ConfigMaps and Secrets for environment-specific configuration.

6 Validation and Results

The repository test suite covers harvester helpers, ABS CPI parsing, GDELT archive processing, GDELT backfill resume behaviour, source registry logic, NLP processing, Redis runtime queue behaviour, analytics store behaviour and API route wiring. The current public branch passes:

53 pytest tests passing

The original cloud run processed approximately 245k raw documents, 225k processed documents and 10,134 ABS CPI observations. These counts are a snapshot of the historical run, not a live service guarantee.

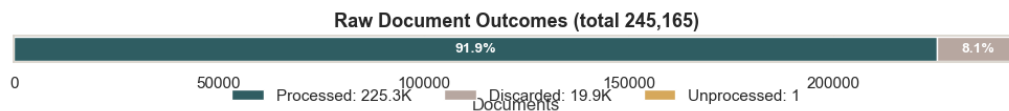


Figure 2: Raw document processing outcomes

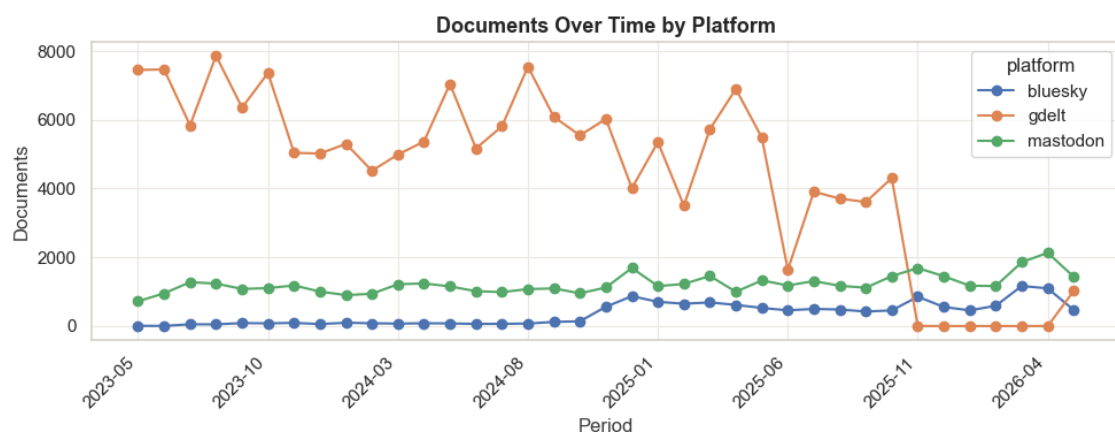


Figure 3: Documents over time by platform

6.1 Topic and Sentiment Patterns

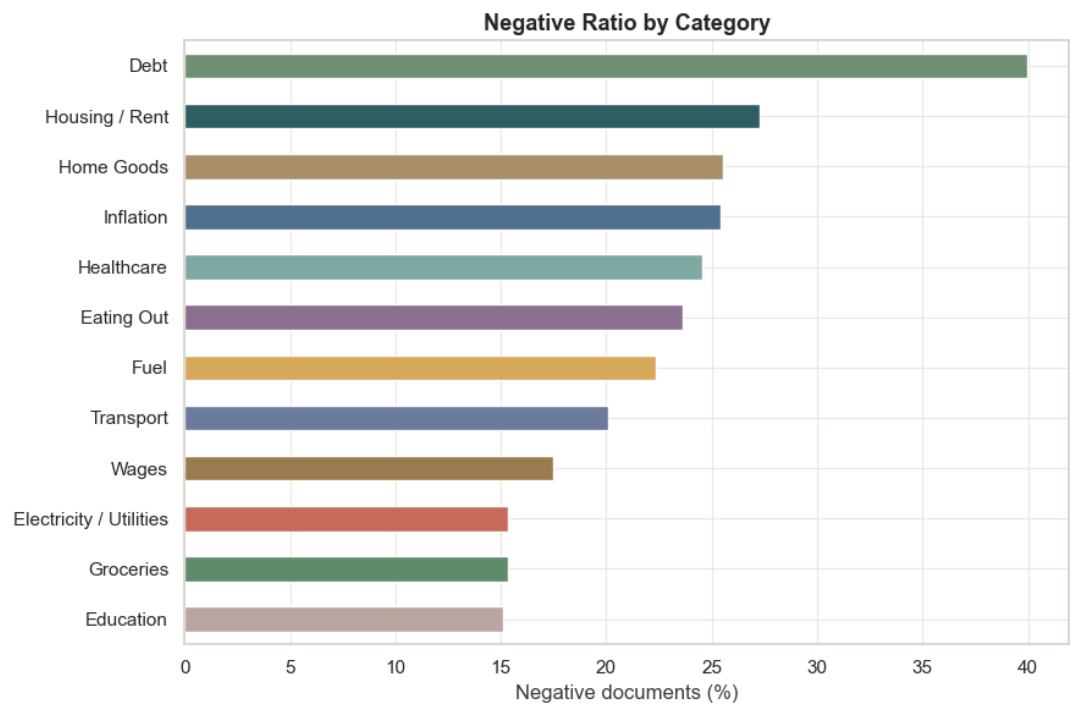


Figure 4: Negative ratio by category

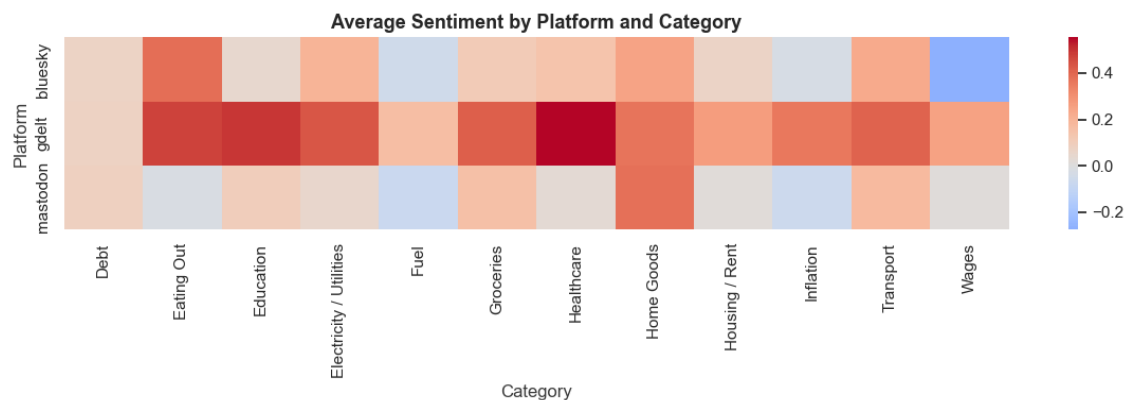


Figure 5: Platform and category sentiment heatmap

The charts show that cost-of-living discussion is not uniform across categories or platforms. Social sources and GDELT should be interpreted separately: social sources are closer to personal expression, while GDELT measures media coverage.

6.2 Official CPI Comparison

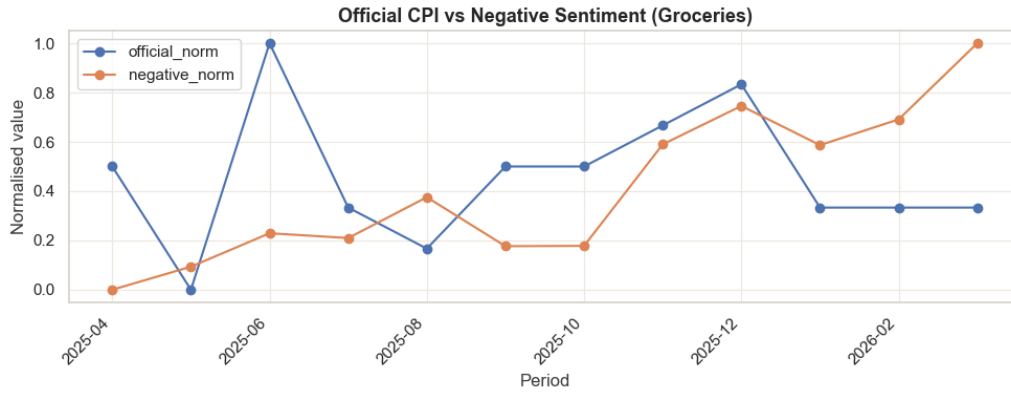


Figure 6: Groceries: official CPI and negative sentiment comparison

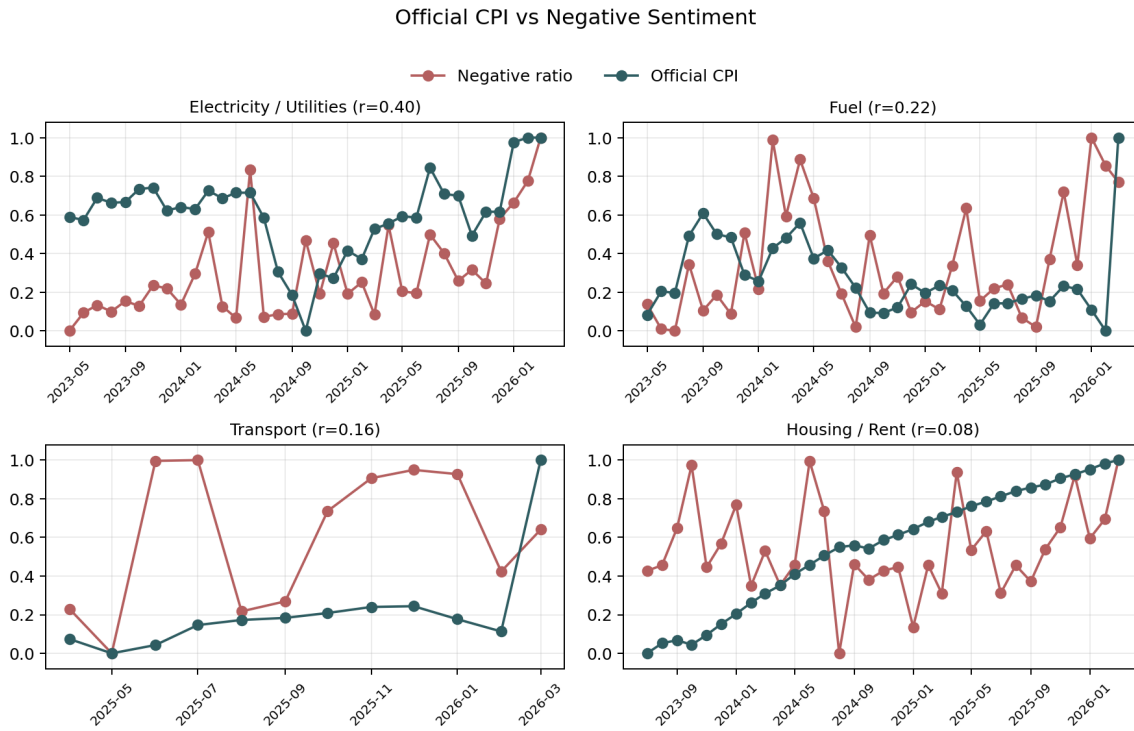


Figure 7: Official CPI and sentiment comparison across selected topics

The CPI comparison is descriptive. CPI measures official price movement, while social and media records measure attention and language. Alignment between the two is useful evidence for interpretation but does not imply causation.

7 Limitations

- Public social platforms are not representative population samples.
- GDELT records can be noisy and metadata-heavy.
- Keyword collection can miss relevant posts that use unexpected wording.
- Topic classification assigns a primary category and can understate cross-topic discussion.

- VADER is transparent and lightweight, but it can misread sarcasm and political language.
- ABS CPI is monthly and delayed relative to online discussion.
- The dataset does not contain reliable city-level fields.

8 Conclusion

The platform demonstrates a maintainable cloud analytics pattern for public text and official indicator data. The strongest engineering choices are the separation of raw and processed data, explicit source registration, archive-based GDELT ingestion for both incremental and historical runs, consistent index configuration, a single API serving path, Redis-backed runtime diagnostics, and a testable FastAPI interface. The main analytical caution is that each source measures a different phenomenon: social discussion, media coverage and official price indicators should be compared carefully rather than merged into one sentiment claim.